

ACPollerForAPS

Invoice pipeline — routing, mapping and delivery

Product Manual

Multi-ERP integration for APS (ReadSoft) invoice exports
Arondor — Activate your content

Contents

1. Introduction & concepts
2. Installation & startup
3. The configuration interface
 - General tab
 - Schedule tab
 - Output channels tab
4. Configuring a channel
5. Field mapping
 - Sources & types
 - Conditional mapping (Vendor/Ledger)
 - Preview
6. Output transports (FS / FTPS / S3)
7. ERP Providers (plugins) — in depth
8. Operations: logs, monitoring, service lifecycle, troubleshooting

1. Introduction & concepts

ACPollerForAPS converts invoices exported from APS (ReadSoft) into the format each destination ERP expects, and delivers them where the ERP imports them. A single input format (APS XML), multiple outputs depending on the Buyer: CSV for Dynamics AX, XML for Optima, and so on — all configuration-driven, with no recompilation.

The pipeline

At a scheduled interval, the service performs the following steps:

- 1. **Scan** the input folder for APS XML files.
- 2. **Route** each file to an output channel, based on its Buyer value.
- 3. **Merge** the files belonging to the same channel (optionally in batches).
- 4. **Convert** to the channel's format (CSV or XML) via its provider.
- 5. **Deliver** the result through the channel transport (FS, FTPS or S3).
- 6. **Archive** the source files — only if delivery succeeded.

Architecture

Component	Role
ACPollerForAPS.Core (DLL)	Shared model, mapping engine, validation, encryption, path extractor, plugin contract. Referenced by both the service and the UI, so there is a single source of truth.
ACPollerForAPS.Service.exe	The Windows service that runs the pipeline.
ACPollerForAPS.UI.exe	The configuration interface (WPF).

The configuration file

Both the service and the UI share a single `settings.json` file (structure `{ "Pipeline": { ... } }`). The service loads it at startup, next to its exe; the UI loads it automatically at launch if present, and edits it.

The service reads `settings.json` once, at startup. After a change through the UI, **restart the service** so it picks up the new configuration.

2. Installation & startup

Prerequisites

- .NET Framework 4.8 (runtime) on the target machine.
- NuGet-restored packages: Newtonsoft.Json, NLog, FluentFTP, AWSSDK.S3, MahApps.Metro (UI).

Installing the service

From an **administrator** command prompt, in the folder containing `ACPollerForAPS.Service.exe`, run `install.bat` :

```
set INSTALLUTIL=%WINDIR%\Microsoft.NET\Framework64\v4.0.30319\InstallUtil.exe
set EXE=%~dp0ACPollerForAPS.Service.exe
"%INSTALLUTIL%" "%EXE%"
sc start ACPollerForAPS
```

Uninstall with `uninstall.bat` (`sc stop` + `InstallUtil /u`).

The service name (`ACPollerForAPS`) in the .bat files must match the ServiceName in the service. Console debug mode: `ACPollerForAPS.Service.exe --console` .

Launching the interface

Run `ACPollerForAPS.UI.exe` . If a `settings.json` sits next to it, it is loaded automatically (shown in the status bar and the title bar).

3. The configuration interface

The window has a branded header, a toolbar (New, Open, Save, Save as, Validate, theme toggle, Help) and three tabs: **General**, **Schedule**, **Output channels**.

General tab

Common folders, file detection and routing paths.

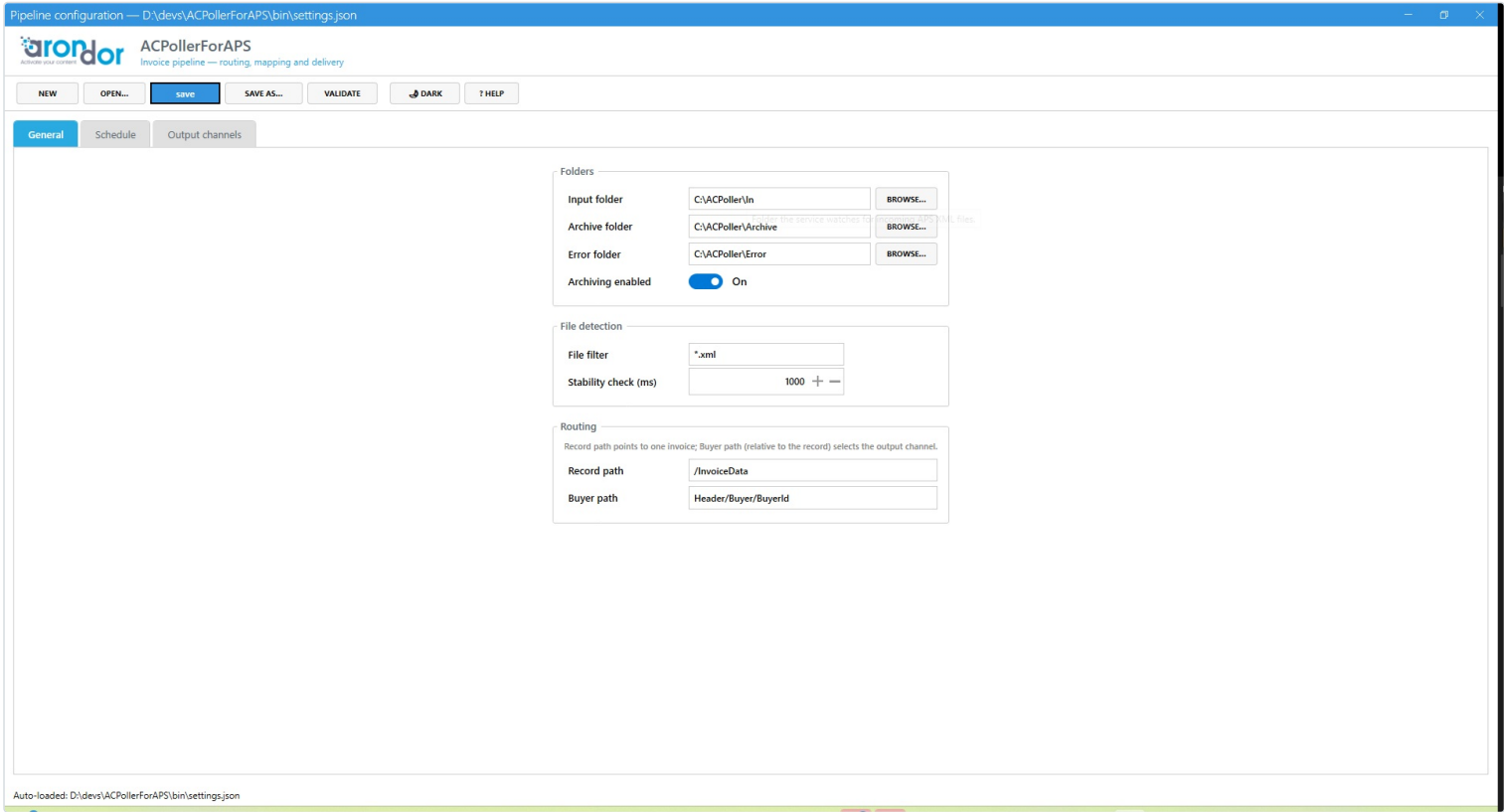


Figure 1 — General tab: folders, file detection, routing.

Field	Meaning
Input folder	Folder the service watches for incoming APS XML files.
Archive folder	Where processed files are moved (dated subfolder) after successful delivery.
Error folder	Where unroutable / unreadable files are moved for review.
Archiving enabled	If off, sources are deleted instead of archived after delivery.
File filter	Which files to pick up (e.g. *.xml).
Stability check (ms)	Delay used to confirm a file is fully written before processing.
Record path	XPath to a single invoice, e.g. /InvoiceData .
Buyer path	XPath (relative to the record) to the value that selects the output channel, e.g. Header/Buyer/BuyerId .

Schedule tab

How often the service wakes up to process and merge files.

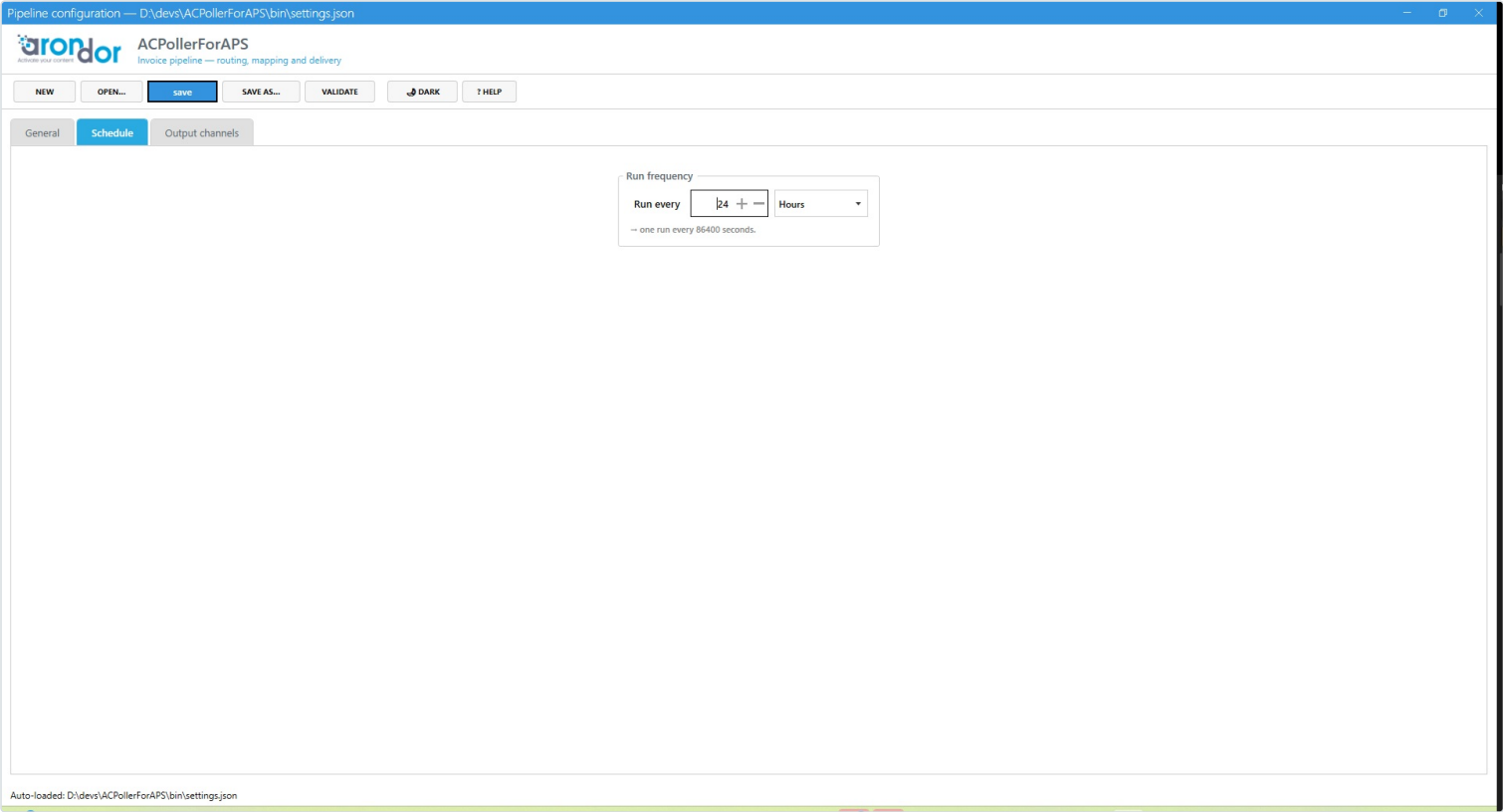


Figure 2 — Schedule tab: run frequency.

A 24h interval means one run per day. For testing, set a few minutes. Note that the run time drifts with the service start time; if a fixed daily time is required, that is a separate scheduling need.

Output channels tab

The channel list is on the left; the selected channel's editor is on the right, with its settings, transport, field mapping and preview.

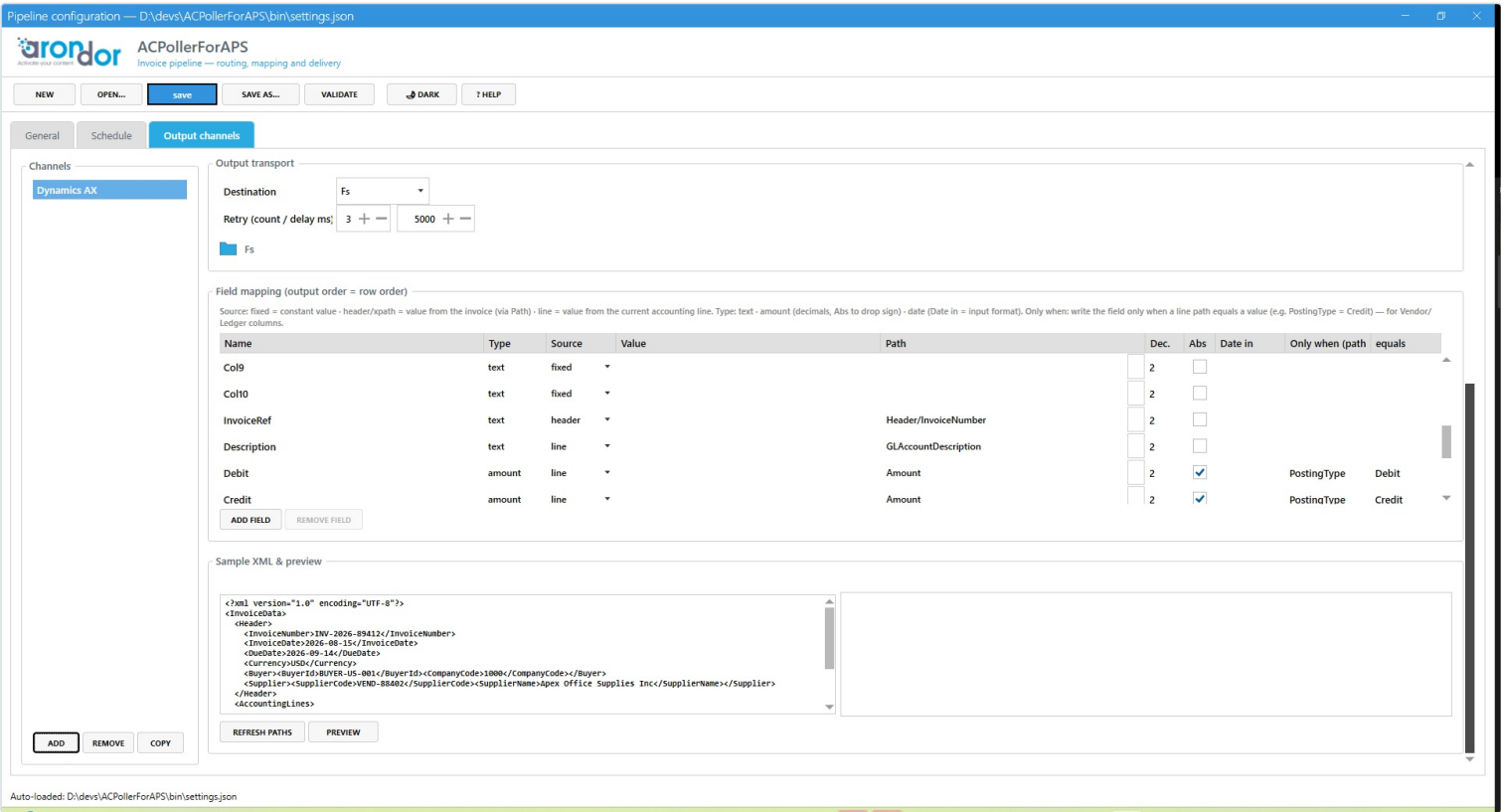


Figure 3 — Output channels tab with the field mapping grid.

4. Configuring a channel

A channel describes one ERP destination: which Buyers it serves, the output format, where and how it is delivered, and how invoices are mapped.

Pipeline configuration — D:\devs\ACPollerForAPS\bin\settings.json

arondor

ACPollerForAPS

Invoice pipeline — routing, mapping and delivery

NEW

OPEN...

SAVE

SAVE AS...

VALIDATE

DARK

HELP

GeneralScheduleOutput channels

Channels

Dynamics AX

ADD

REMOVE

COPY

Channel settings

Channel name

Dynamics AX

Enabled

On

Buyers (semicolon)

BUYER-US-001;BUYER-US-002

Output format

Csv

Output folder

C:\ACPoller\Out\DynamicsAX

Output file name

{date}.csv

Record path

/InvoiceData

Lines path

AccountingLines/AccountingLine

Provider

mapping

Sample XML

LOAD SAMPLE XML...

loaded (1291 chars)

Invoices per file

0

+

-

0 = one merged file

Output transport

Destination

Fs

Retry (count / delay ms)

3

+

-

5000

+

-

Fs

Field mapping (output order = row order)

Source: fixed = constant value - header/x/path = value from the invoice (via Path) - line = value from the current accounting line. Type: text - amount (decimals, Abs to drop sign) - date (Date in = input format). Only when: write the field only when a line path equals a value (e.g. PostingType = Credit) — for Vendor/ Ledger columns.

Name	Type	Source	Value	Path	Dec.	Abs	Date in	Only when (path equals
Col9	text	fixed			2	☐		
Col10	text	fixed			2	☐		

Auto-loaded: D:\devs\ACPollerForAPS\bin\settings.json

Figure 4 — Channel settings.

Field	Meaning
Channel name / Enabled	A label; a disabled channel is ignored by the pipeline.
Buyers (semicolon)	Buyer values routed to this channel, separated by <code>;</code> .
Output format	Csv or Xml.
Output folder / file name	Destination and name pattern. Tokens: <code>{date}</code> <code>{time}</code> <code>{guid}</code> <code>{batch}</code> .
Record path / Lines path	Location of the invoice and of the repeating accounting lines.
Provider	<code>mapping</code> (default) or a plugin DLL name — see section 7.
Sample XML	A representative input XML, stored in the config. Feeds both the preview and the Path auto-completion.
Invoices per file	Max invoices per output file for this channel. <code>0</code> = all invoices in one merged file. E.g. 5 with 30 invoices ⇒ 6 files of 5.

5. Field mapping

The mapping grid defines the output columns (CSV) or elements (XML), in order. Each row is one output field.

Sources & types

Property	Meaning
Name	Output column / element name.
Type	<code>text</code> · <code>amount</code> (decimals; <i>Abs</i> drops the sign) · <code>date</code> (<i>Date in</i> = input format).
Source	<code>fixed</code> = constant (Value) · <code>header</code> / <code>xpath</code> = a node of the invoice (Path) · <code>line</code> = a node of the current accounting line (Path).
Value	Constant, when Source = fixed.
Path	XPath. The suggestion list is filtered by Source.
Dec. / Abs	Decimals / absolute value, for amounts.
Date in	Input date format (dates are reformatted to the channel's output format).
Only when (path) / equals	Conditional write — see below.

Conditional mapping (Vendor / Ledger lines)

Some formats (Dynamics AX) produce, per invoice, one **Vendor** line (the vendor credit) and N **Ledger** lines (the expense debits), where some columns differ depending on the line type. The **Only when (path) / equals** columns handle this: a field is written only when a node of the line equals a value; otherwise the cell is empty.

Typical Dynamics setup:

Column	Only when (path)	equals	Effect
Credit	PostingType	Credit	amount only on the Vendor line
Debit	PostingType	Debit	amount only on Ledger lines
Vendor control account	PostingType	Credit	only on the Vendor line

This is how a single invoice (one Credit line + several Debit lines in the XML) becomes one Vendor row and several Ledger rows in the CSV, each with the right columns filled.

Preview

Load a *Sample XML* in the channel, then click **Preview** to see the generated output. **Refresh paths** re-scans the sample XML to update the Path suggestions. The **Validate** button (toolbar) checks the whole configuration and reports errors and warnings before saving.

6. Output transports

Each channel delivers its file through a transport, chosen in the **Output transport** section. A configurable retry (count / delay) applies; if delivery fails after all attempts, the generated file is queued for later delivery (see below) — never lost.

Failed deliveries & automatic retry

When a delivery fails after all retries (e.g. the FTPS server or S3 endpoint is unreachable), the pipeline does **not** lose anything:

- The **generated output file** (the CSV/XML already produced) is moved to a **pending queue** (`pending/<channel>/`), with a companion file recording its target channel.
- The source XML files are **archived normally** — the generated content is safe in the queue, so nothing is at risk.
- At the **start of every run**, before processing new files, the service retries the queued deliveries. Files that go through leave the queue; the others stay for the next attempt.

A failed delivery is therefore **retried automatically** and the file is delivered — the exact file that was generated — as soon as the server is reachable again, in chronological order (queued files go before new ones). Pending deliveries appear in the run summary and raise a warning in the Windows Event Log.

FS — file system

The simplest transport: the file is written to the channel's Output folder.

FTPS

Delivery to an FTPS server (explicit / implicit mode, optional certificate validation). The password is encrypted and never stored in clear.

Pipeline configuration — D:\devs\ACPollerForAPS\bin\settings.json

arondor

ACPollerForAPS

Invoice pipeline — routing, mapping and delivery

NEW

OPEN...

SAVE

SAVE AS...

VALIDATE

DARK

HELP

General

Schedule

Output channels

Channels

Dynamics AX

Record path

/InvoiceData

Lines path

AccountingLines/AccountingLine

Provider

mapping

Sample XML

LOAD SAMPLE XML...

loaded (1291 chars)

Invoices per file

0

0 = one merged file

Output transport

Destination

Ftps

Retry (count / delay ms)

3

5000

Ftps

Host

ftp.example.com

Port

21

Remote folder

/

Username

Password

leave empty to keep current

(stored encrypted)

FTPS mode

Explicit

☒ Validate certificate

TEST CONNECTION

Field mapping (output order = row order)

Source: fixed = constant value - header/xpath = value from the invoice (via Path) - line = value from the current accounting line. Type: text = amount (decimals, Abs to drop sign) - date (Date in = input format). Only when: write the field only when a line path equals a value (e.g. PostingType = Credit) — for Vendor/ Ledger columns.

Name	Type	Source	Value	Path	Dec.	Abs	Date in	Only when (path equals
Col9	text	fixed			2	☐		
Col10	text	fixed			2	☐		

ADD

REMOVE

COPY

Auto-loaded: D:\devs\ACPollerForAPS\bin\settings.json

Figure 5 — FTPS transport settings.

Field	Meaning
Host / Port	FTPS server; 21 (explicit) or 990 (implicit).
Remote folder	Target folder on the server (created if missing).
Username / Password	Credentials; password stored encrypted (leave empty to keep the current one).
FTPS mode	Explicit (FTPES, AUTH TLS) or Implicit (TLS from the start).
Validate certificate	Verify the server TLS certificate (turn off only for test servers).
Test connection	Opens a connection with these settings to verify host, credentials and TLS.

S3 (and S3-compatible / MinIO)

Delivery to an Amazon S3 bucket, or any S3-compatible store via a custom endpoint.

Pipeline configuration — D:\devs\ACPollerForAPS\bin\settings.json

arondor

Activate your content

ACPollerForAPS

Invoice pipeline — routing, mapping and delivery

NEW

OPEN...

save

SAVE AS...

VALIDATE

DARK

? HELP

General

Schedule

Output channels

Channels

Dynamics AX

Record path

/InvoiceData

Lines path

AccountingLines/AccountingLine

Provider

mapping

Sample XML

LOAD SAMPLE XML...

loaded (1291 chars)

Invoices per file

0

0 = one merged file

Output transport

Destination

S3

Retry (count / delay ms)

3

5000

S3

Bucket

Key prefix

optional, e.g. dynamics/

Region

eu-west-1

Endpoint (custom)

leave empty for AWS; e.g. https://minio.local:9000

Path-style access

☐

Access key

Secret key

leave empty to keep current

(stored encrypted)

TEST CONNECTION

Field mapping (output order = row order)

Source: fixed = constant value - header/xpath = value from the invoice (via Path) - line = value from the current accounting line. Type: text - amount (decimals, Abs to drop sign) - date (Date in = input format). Only when: write the field only when a line path equals a value (e.g. PostingType = Credit) — for Vendor/ Ledger columns.

Name	Type	Source	Value	Path	Dec.	Abs	Date in	Only when (path equals
Col9	text	fixed			2	☐		

ADD

REMOVE

COPY

Auto-loaded: D:\devs\ACPollerForAPS\bin\settings.json

Figure 6 — S3 transport settings.

Field	Meaning
Bucket / Key prefix	Target bucket; optional logical folder prepended to the file name.
Region	AWS region (ignored if a custom endpoint is set).
Endpoint (custom)	For MinIO / S3-compatible; empty = real AWS S3.
Path-style access	Required by MinIO and many compatible servers.
Access key / Secret key	Credentials; secret stored encrypted.
Test connection	Checks the bucket is reachable with these credentials.

Credential security

FTPS passwords and S3 secret keys are encrypted with **DPAPI (machine scope)** — never stored in clear in the JSON.

Important: secrets must be encrypted *on the target machine* (the one running the service). A secret encrypted on another machine cannot be decrypted there. Use the UI installed on the service machine to enter passwords.

7. ERP Providers (plugins) advanced

The provider mechanism is what makes the product extensible to any ERP without touching the core. It follows a **hybrid** approach.

Two levels

Level	When	How
Mapping (default)	Most ERPs — output describable by the field mapping (CSV / XML).	No DLL. Set Provider = mapping and configure the mapping grid.
DLL plugin	An ERP with specific logic (aggregations, unusual structure, business rules) the mapping cannot express.	Ship a DLL implementing IErpExporter in the providers/ folder; set Provider to its name.

How the service discovers providers

At startup, the service:

- registers the built-in **mapping** provider;
- scans **<exe folder>/providers/*.dll**;
- instantiates every type implementing **IErpExporter** and registers it under the name(s) it declares.

A DLL that is unreadable, incompatible, or whose type fails to load is **logged and ignored** — it never crashes the service. If a channel references an unknown provider name, the service falls back to **mapping** and logs it.

The contract: IErpExporter

Defined in the shared **ACPollerForAPS.Core** DLL:

```
public class ExportResult
{
    public byte[] Content { get; set; }      // file ready to deliver
    public List<string> Warnings { get; }     // non-blocking warnings
}

public interface IErpExporter
{
    // Name(s) this provider registers under (referenced by
    // OutputChannel.Provider). Case-insensitive.
    IEnumerable<string> ProviderNames { get; }

    // Transforms the input XMLs (already routed to this channel) into a
    // single output file, according to the channel configuration.
    ExportResult Export(IEnumerable<string> inputXmls, OutputChannel channel);
}
```

Building a provider

- New **class library** (.NET 4.8) referencing **ACPollerForAPS.Core**.
- Implement **IErpExporter**:

```
using System.Collections.Generic;
using System.Text;
using ACPollerForAPS.Core;

public class OptimaExporter : IErpExporter
{
    public IEnumerable<string> ProviderNames => new[] { "optima" };

    public ExportResult Export(IEnumerable<string> inputXmls, OutputChannel channel)
    {
        var result = new ExportResult();
        // ... your specific transformation from inputXmls ...
        // (channel gives RecordPath, LinesPath, Fields, formats, etc.)
        result.Content = Encoding.UTF8.GetBytes(/* the produced file */);
        return result;
    }
}
```

- Compile the DLL.
- Drop it in the **providers/** folder next to **ACPollerForAPS.Service.exe** (create the folder if needed).
- Set **Provider** to its name on the relevant channel (via the UI or the JSON).

No recompilation of the service. Because the contract lives in the shared Core DLL, the provider sees exactly the same model as the service and the UI.

Version compatibility: a provider is compiled against a version of **ACPollerForAPS.Core**. The Core DLL shipped next to the service must be compatible. If the Core contract changes, recompile the providers.

What stays common

A provider only produces the file content. Everything else is handled by the pipeline, identically for every provider: input detection and reading, Buyer routing, batching, output

file naming, transport (FS/FTPS/S3) with retry, and archiving on success.

8. Operations

Logs

NLog writes to `logs/` next to the service exe, with daily + size rotation and automatic purge (30 archives). Each run logs a start line, per-file routing detail, and a summary (delivered, per batch, errors). For detailed diagnostics, set the log level to `Debug` in `NLog.config` (no recompilation), then switch back to `Info`.

Monitoring (Windows Event Log)

Beyond the file logs, the service reports to a dedicated Windows event log named **ACPollerForAPS** (visible in the Event Viewer). This is the standard, agent-less way to supervise the service: an operator can watch it manually today, and any enterprise monitoring tool (Nagios, Zabbix, Centreon...) reads the Windows Event Log natively.

What is written

- **Business events** — service start / stop, and a per-run summary (files delivered, outputs written, errors), so a glance at the Event Viewer is enough to judge the service health.
- **All warnings and errors** — any incident logged by the service is also sent to the Event Log (safety net).

Events are graded by severity: *Information* (normal operation), *Warning* (non-blocking anomalies), *Error* (failures). Filter by severity in the Event Viewer for a quick health check.

Event IDs

ID	Severity	Meaning
1000	Information	Service started
1001	Information	Service stopped
1100	Information	Run completed OK (with the run summary)
1200	Warning	Run completed with anomalies (delivery failures, errors, unroutable files)
1300	Error	Delivery failure

Where to look

Event Viewer → *Applications and Services Logs* → **ACPollerForAPS**. Or from PowerShell:

```
Get-EventLog -LogName ACPollerForAPS -Newest 20
```

Source creation

The event source is created **at service installation** (which runs as administrator). No manual step is required. If the source is missing (partial install), the service degrades gracefully: it keeps writing to the file log and disables the Event Log output, without crashing. To create the source, reinstall the service as administrator.

After enabling this feature, **reinstall the service** (`uninstall.bat` then `install.bat`) so the event source is created — a simple restart is not enough.

Connecting an enterprise monitor

Point the monitoring tool at the **ACPollerForAPS** Windows event log. Recommended rules: alert on any *Error*; alert on *Warning* ID 1200 (runs with anomalies); optionally alert if no ID 1100/1200 has appeared within the expected run interval (the service is not running or not processing).

Service lifecycle

```
sc start ACPollerForAPS
sc pause ACPollerForAPS
sc continue ACPollerForAPS
sc stop ACPollerForAPS
```

Remember to restart the service after changing `settings.json`.

Troubleshooting

Symptom	Likely cause
FileNotFoundException at install	The .bat points to the wrong exe name, or is launched outside the exe folder.
<code>[SC] OpenService 1060</code>	Install failed earlier; the service does not exist.
"the service did not respond in time"	ServiceName mismatch between the code and the .bat.
Service runs but processes nothing	Interval too long (e.g. 24h — it waits for its slot), wrong input folder, or no channel matches the Buyer. Check the logs.
FTPS/S3 secret cannot be decrypted	Encrypted on a machine other than the service's (DPAPI machine scope).
Files stuck in <code>pending/</code>	Delivery keeps failing (server unreachable, bad credentials). Files are safe and retried each run; check the transport settings and the Event Log.

Business items to finalize

- Real APS input XMLs to lock the final mapping.
- Dynamics CSV columns still unmapped (AP codes, MCC dimensions, intermediate columns).
- Optima output format.